# Chapter 10 (Multiprocessor Scheduling) — Homework (Simulation)

Zhiyuan Pan

All runs use `multi.py` in `ostep-homework/cpu-sched-multi/`. Every command below can be rerun with `code/chapter10/run.sh`.

How the simulator models caches (from reading `multi.py`):

- Each job has a **run time** and a **working-set size**; each CPU has a cache of size `-M` (default 100). Jobs are given with `-L name:runtime:wset,...`.
- A job running on a CPU whose cache is **cold** for it finishes 1 unit of work per tick. After it has run `-w` ticks (default 10) on that CPU, the cache becomes **warm** for it, and from the next tick on it does `-r` units per tick (default 2).
- When a job becomes warm, it is put at the front of the cache's list, and jobs are evicted from the back until the total working set fits in `-M`. A job whose working set is larger than the cache therefore evicts everything, including itself, and is never warm.
- The default policy is one **central queue**, round robin with a 10-tick time slice (`-q`). At every slice boundary, all running jobs go back to the end of the queue, and idle CPUs (in CPU order) take the first job they are allowed to run.
- **Note:** the default number of CPUs is **2** (`-n`), not 1.

---

## Q1

> To start things off, let's learn how to use the simulator to study how to build an effective multi-processor scheduler. The first simulation will run just one job, which has a run-time of 30, and a working-set size of 200. Run this job (called job 'a' here) on one simulated CPU as follows: `./multi.py -n 1 -L a:30:200`. How long will it take to complete? Turn on the `-c` flag to see a final answer, and the `-t` flag to see a tick-by-tick trace of the job and how it is scheduled.

**It takes 30 ticks.** The working set (200) is larger than the cache (100), so the cache can never hold it. Even when the 10-tick warm-up finishes, the simulator immediately evicts the job again. Job `a` runs cold the whole time, at 1 unit of work per tick, so 30 units take 30 ticks.

```
$ ./multi.py -n 1 -L a:30:200 -c -t
   0   a
   1   a
   ...
  29   a
Finished time 30
  CPU 0  utilization 100.00 [ warm 0.00 ]
```

The `-t` trace simply shows `a` on CPU 0 on every tick (0–29), and the stats confirm it was never warm (`warm 0.00`). With `-C` added, the cache column never shows a `w`.

## Q2

> Now increase the cache size so as to make the job's working set (size=200) fit into the cache (which, by default, is size=100); for example, run `./multi.py -n 1 -L a:30:200 -M 300`. Can you predict how fast the job will run once it fits in cache? (hint: remember the key parameter of the warm rate, which is set by the `-r` flag) Check your answer by running with the solve flag (`-c`) enabled.

**Prediction: 20 ticks.** For the first 10 ticks (the warm-up time) the cache is cold, so the job does 10 units of work. Then the cache is warm and the job runs at the warm rate of 2 units per tick: the remaining 20 units take 10 ticks. Total: 10 + 20/2 = **20**.

```
$ ./multi.py -n 1 -L a:30:200 -M 300 -c
Finished time 20
  CPU 0  utilization 100.00 [ warm 50.00 ]
```

Correct: half of the 20 ticks ran warm. In general, for one job of run time R on one CPU, with warm-up time w < R and warm rate r:

$$T = w + \left\lceil \frac{R - w}{r} \right\rceil$$

Checked by changing the warm rate:

| `-r` | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| finish time | 30 | 20 | 17 | 15 |
| formula | 10 + 20/1 = 30 | 10 + 20/2 = 20 | 10 + ⌈20/3⌉ = 17 | 10 + 20/4 = 15 |

## Q3

> One cool thing about `multi.py` is that you can see more detail about what is going on with different tracing flags. Run the same simulation as above, but this time with time left tracing enabled (`-T`). This flag shows both the job that was scheduled on a CPU at each time step, as well as how much run-time that job has left after each tick has run. What do you notice about how that second column decreases?

```
$ ./multi.py -n 1 -L a:30:200 -M 300 -c -T
   0   a [ 29]
   1   a [ 28]
   ...
   9   a [ 20]
----------------
  10   a [ 18]
  11   a [ 16]
  ...
  19   a [  0]
```

The time left **decreases by 1 per tick for the first 10 ticks** (29, 28, …, 20), while the cache is cold, and then **by 2 per tick** (18, 16, …, 0) once the cache is warm. The change happens exactly after the warm-up time of 10 ticks, which is why the job finishes at 20 instead of 30. (The dashed line is the end of a 10-tick time slice; with only one job, the same job is simply scheduled again on the same CPU.)

## Q4

> Now add one more bit of tracing, to show the status of each CPU cache for each job, with the `-C` flag. For each job, each cache will either show a blank space (if the cache is cold for that job) or a 'w' (if the cache is warm for that job). At what point does the cache become warm for job 'a' in this simple example? What happens as you change the warmup time parameter (`-w`) to lower or higher values than the default?

```
$ ./multi.py -n 1 -L a:30:200 -M 300 -c -T -C
   8   a [ 21] cache[ ]
   9   a [ 20] cache[w]
----------------------
  10   a [ 18] cache[w]
```

**The cache becomes warm at the end of tick 9**, i.e. after job `a` has run for 10 ticks (the default `-w 10`). The `w` first appears on the tick-9 line, because the cache column is printed after that tick's warm-up update. Tick 9 itself still ran at the cold rate (time left 21 → 20); tick 10 is the first warm tick (20 → 18).

Changing the warm-up time (`-M 300` so the job fits):

| `-w` | 0 | 1 | 5 | 10 | 15 | 20 | 25 | 30 | 40 |
|---|---|---|---|---|---|---|---|---|---|
| finish time | 15 | 16 | 18 | 20 | 23 | 25 | 28 | 30 | 30 |
| ticks run warm | 100% | 93.8% | 72.2% | 50% | 34.8% | 20% | 10.7% | 0% | 0% |

- A **shorter** warm-up helps: the job reaches the fast rate sooner. With `-w 0` the cache is warm immediately (the simulator special-cases 0), so all 30 units run at rate 2: 15 ticks.
- A **longer** warm-up hurts. Once `w` ≥ 30 (the job's run time), the job finishes before its cache ever becomes useful, and it takes the full 30 ticks, the same as when it did not fit in the cache at all.
- Every value matches T = w + ⌈(30 − w)/2⌉ from Q2.

## Q5

> At this point, you should have a good idea of how the simulator works for a single job running on a single CPU. But hey, isn't this a multi-processor CPU scheduling chapter? Oh yeah! So let's start working with multiple jobs. Specifically, let's run the following three jobs on a two-CPU system (i.e., type `./multi.py -n 2 -L a:100:100,b:100:50,c:100:50`) Can you predict how long this will take, given a round-robin centralized scheduler? Use `-c` to see if you were right, and then dive down into details with `-t` to see a step-by-step and then `-C` to see whether caches got warmed effectively for these jobs. What do you notice?

**Prediction: 150 ticks.** The total work is 300 units on 2 CPUs. If the caches never help, that takes 300/2 = 150 ticks. I expect them not to help, because three jobs share two CPUs round-robin, so jobs keep moving between CPUs.

```
$ ./multi.py -n 2 -L a:100:100,b:100:50,c:100:50 -c -t -T -C
   0   a [ 99] cache[   ]     b [ 99] cache[   ]
   ...
   9   a [ 90] cache[w  ]     b [ 90] cache[ w ]
------------------------------------------------
  10   c [ 99] cache[w  ]     a [ 89] cache[ w ]
   ...
  19   c [ 90] cache[  w]     a [ 80] cache[w  ]
------------------------------------------------
  20   b [ 89] cache[  w]     c [ 89] cache[w  ]
...
Finished time 150
  CPU 0  utilization 100.00 [ warm 0.00 ]
  CPU 1  utilization 100.00 [ warm 0.00 ]
```

Correct: 150 ticks, and **not a single tick ran warm** on either CPU. The caches were warmed, but always uselessly:

- With 3 jobs and 2 CPUs, the central queue hands each job to a **different CPU in every time slice**: CPU 0 runs a, c, b, a, c, …; CPU 1 runs b, a, c, b, …
- The warm-up time (10) equals the time slice (10). A job becomes warm on a CPU exactly at the last tick of its slice (the `w` appears on ticks 9, 19, 29, …) and is then moved to the other CPU, where it is cold again.

- On top of that, job `a`'s working set (100) fills an entire cache, so whenever `a` warms up on a CPU, it evicts `b` and `c` there (e.g. CPU 0 goes from `[ ww]` back to `[w  ]` at tick 39).

So a single central queue has no **cache affinity**: it balances load perfectly (both CPUs 100% busy) but throws away all the benefit of the caches.

## Q6

> Now we'll apply some explicit controls to study cache affinity, as described in the chapter. To do this, you'll need the `-A` flag. This flag can be used to limit which CPUs the scheduler can place a particular job upon. In this case, let's use it to place jobs 'b' and 'c' on CPU 1, while restricting 'a' to CPU 0. This magic is accomplished by typing this `./multi.py -n 2 -L a:100:100,b:100:50,c:100:50 -A a:0,b:1,c:1`; don't forget to turn on various tracing options to see what is really happening! Can you predict how fast this version will run? Why does it do better? Will other combinations of 'a', 'b', and 'c' onto the two processors run faster or slower?

**Prediction: 110 ticks.**

- **CPU 0** runs only `a`: 10 cold ticks (10 units), then 90 units at rate 2 = 45 ticks. `a` finishes at **55**.
- **CPU 1** runs `b` and `c` alternately. Their working sets (50 + 50) fit together in the 100-unit cache. Each needs one cold slice: b during 0–10, c during 10–20. After that, both stay warm. The remaining 90 + 90 = 180 units at rate 2 take 90 ticks, so CPU 1 finishes at 20 + 90 = **110**.

```
$ ./multi.py -n 2 -L a:100:100,b:100:50,c:100:50 -A a:0,b:1,c:1 -c -t -T -C
   9   a [ 90] cache[w  ]     b [ 90] cache[ w ]
  10   a [ 88] cache[w  ]     c [ 99] cache[ w ]
  19   a [ 70] cache[w  ]     c [ 90] cache[ ww]
  20   a [ 68] cache[w  ]     b [ 88] cache[ ww]
  54   a [  0] cache[w  ]     c [ 60] cache[ ww]
  63   - [   ] cache[w  ]     b [ 42] cache[ ww]
 109   - [   ] cache[w  ]     c [  0] cache[ ww]
Finished time 110
  CPU 0  utilization 50.00 [ warm 40.91 ]
  CPU 1  utilization 100.00 [ warm 81.82 ]
```

Correct: **110 ticks instead of 150.** It does better because of **cache affinity**. Each job always runs on the same CPU, and the jobs on each CPU fit in its cache together, so after a short warm-up almost everything runs at double speed (82% of CPU 1's ticks are warm). This more than makes up for the load imbalance: CPU 0 is idle after `a` finishes at 55 (50% utilization).

**Other combinations are slower:**

| `-A` | CPU 0 | CPU 1 | finish time |
|---|---|---|---|
| `a:0,b:1,c:1` | a | b, c | **110** |
| `b:0,a:1,c:1` | b | a, c | 200 |
| `c:0,a:1,b:1` | c | a, b | 200 |
| `a:0,b:0,c:1` | a, b | c | 200 |
| `a:0.1,b:1,c:1` (a may use either CPU) | a | b, c | 110 |
| no affinity (Q5) | rotating | rotating | 150 |

Any placement that puts `a` together with another job on the same CPU is bad: `a` (100) plus `b` or `c` (50) needs 150 units of cache, more than the 100 available. The two jobs keep evicting each other and run cold all the time, so that CPU needs 200 ticks for 200 units of work, worse even than the central queue. The only good split is the one that keeps each CPU's total working set within its cache: `a` alone, `b` and `c` together.

## Q7

> One interesting aspect of caching multiprocessors is the opportunity for better-than-expected speed up of jobs when using multiple CPUs (and their caches) as compared to running jobs on a single processor. Specifically, when you run on N CPUs, sometimes you can speed up by more than a factor of N, a situation entitled super-linear speedup. To experiment with this, use the job description here (`-L a:100:100,b:100:100,c:100:100`) with a small cache (`-M 50`) to create three jobs. Run this on systems with 1, 2, and 3 CPUs (`-n 1`, `-n 2`, `-n 3`). Now, do the same, but with a larger per-CPU cache of size 100. What do you notice about performance as the number of CPUs scales? Use `-c` to confirm your guesses, and other tracing flags to dive even deeper.

Three jobs, each with 100 units of work and a working set of 100:

| cache `-M` | 1 CPU | 2 CPUs | 3 CPUs | speedup with 3 CPUs |
|---|---|---|---|---|
| 50 | 300 | 150 | 100 | 3.0× (linear) |
| 100 | 300 | 150 | **55** | **5.45×** (super-linear) |

- **Cache 50:** no job's working set (100) ever fits, so everything always runs cold. The time is just total work / CPUs: 300, 150, 100. The speedup is exactly linear.
- **Cache 100, 1 or 2 CPUs:** each working set exactly fills one cache, but with more jobs than CPUs the jobs rotate between CPUs (and evict each other) every time slice, as in Q5. Nothing ever runs warm: 300 and 150 again.
- **Cache 100, 3 CPUs:** now each job has its own CPU **and** its own cache. The central queue keeps giving every job the same CPU (all three are requeued and taken in the same order each slice). Each

job runs 10 cold ticks and then 90 units at double speed: 10 + 45 = **55 ticks**, with 82% of all ticks warm.

```
$ ./multi.py -n 3 -L a:100:100,b:100:100,c:100:100 -M 100 -c -t -C
   9   a cache[w  ]     b cache[ w ]     c cache[  w]
  10   a cache[w  ]     b cache[ w ]     c cache[  w]
Finished time 55
```

Going from 1 to 3 CPUs makes the jobs **5.45 times faster**, more than the 3× you would expect from adding CPUs. The extra speed comes from the extra **cache capacity**: three CPUs bring three caches, which together are finally big enough to hold all three working sets at once. That is super-linear speedup. It only happens when the per-CPU cache is big enough for a job's working set; with `-M 50` the extra caches do not help at all.

## Q8

> One other aspect of the simulator worth studying is the per-CPU scheduling option, the `-p` flag. Run with two CPUs again, and this three job configuration (`-L a:100:100,b:100:50,c:100:50`). How does this option do, as opposed to the hand-controlled affinity limits you put in place above? How does performance change as you alter the 'peek interval' (`-P`) to lower or higher values? How does this per-CPU approach work as the number of CPUs scales?

With `-p`, each CPU has its own queue. Jobs are dealt out to the queues round robin (here CPU 0: `a, c`; CPU 1: `b`). Every `-P` ticks (default 30), a CPU whose queue is **empty** picks another CPU at random and steals one job from its queue.

```
$ ./multi.py -n 2 -L a:100:100,b:100:50,c:100:50 -p -c
Scheduler CPU 0 queue: ['a', 'c']
Scheduler CPU 1 queue: ['b']
Finished time 100
  CPU 0  utilization 95.00 [ warm 35.00 ]
  CPU 1  utilization 95.00 [ warm 75.00 ]
```

**Per-CPU queues finish in 100 ticks: better than both the central queue (150) and my hand-made affinity (110).** The jobs mostly stay on one CPU, so they keep their caches warm (affinity). Work stealing also fixes the load imbalance that hurt Q6: when one CPU runs out of work, it takes a job from the other instead of sitting idle.

**Peek interval:**

| **-P** | **0 (no stealing)** | **1** | **2** | **5** | **10** | **20** | **25** | **30** | **40** | **50** | **100** |
|---|---|---|---|---|---|---|---|---|---|---|---|
| finish time | 200 | **90** | **90** | **90** | 100 | 100 | 105 | 100 | 115 | 130 | 130 |

- `-P 0` **(never steal) is the worst case, 200.** `a` and `c` are stuck together on CPU 0, their working sets (150) do not fit, and they thrash. Meanwhile CPU 1 is idle after `b` finishes (27.5% utilization).
- **Small intervals (1–5) give the best result, 90.** At the very first peek, CPU 1's queue is empty (b is running), so it steals `c`. That produces the ideal split from Q6 (a | b, c) within one tick. Later, when `a` finishes on CPU 0 at 55, CPU 0 steals `b`, so both CPUs keep working until the end. This beats static affinity (110), which left CPU 0 idle for the last 55 ticks.
- **Large intervals (40–100) get worse, 115–130**, because the imbalance is noticed and fixed later, so `a` and `c` thrash on CPU 0 for longer.
- The trend is not perfectly smooth (25 → 105, 30 → 100), because it matters where each peek falls relative to the time slices and to when jobs finish.

Stealing more often costs nothing in this simulator. In a real system each peek means touching another CPU's queue (locks, cache-line traffic), so very small intervals have a real cost. This is the trade-off the chapter describes.

**Scaling the number of CPUs:**

| CPUs | 3 jobs: central | 3 jobs: per-CPU | 6 jobs: central | 6 jobs: per-CPU (P=30) | 6 jobs: per-CPU (P=5) |
|---|---|---|---|---|---|
| 1 | 300 | 300 | 600 | 600 | (crash, see below) |
| 2 | 150 | **100** | 300 | 300 | 300 |
| 3 | 55 | 55 | **140** | 170 | 150 |
| 4 | **55** | 60 | 150 | 125 | **120** |
| 6 | — | — | 55 | 55 | 55 |

(6 jobs = the same three jobs twice: `a,d` with working set 100 and `b,c,e,f` with 50.)

- When there are **more jobs than CPUs**, per-CPU queues usually win (3 jobs on 2 CPUs: 100 vs 150; 6 jobs on 4 CPUs: 120–125 vs 150), because the central queue keeps migrating jobs and never lets a cache warm up.
- When there are **as many or more CPUs than jobs**, per-CPU queues can lose. With 3 jobs on 4 CPUs, CPU 3 has an empty queue from the start and keeps stealing jobs away from CPUs where they were already warm: 60 (P=30) or 70 (P=5) vs 55 for the central queue. Stealing is only useful when there is an imbalance to fix.
- The result also depends on how the initial round-robin assignment happens to group the working sets (6 jobs on 3 CPUs: per-CPU puts `a` and `d` together on CPU 0, where they thrash, so it loses 170 vs 140).
- With more than 2 CPUs, the victim CPU is chosen at random, so the result depends on the seed (6 jobs, 4 CPUs, P=5: 100–120 over seeds 0–4).
- More CPUs is not always faster with a central queue: 6 jobs took 140 ticks on 3 CPUs but 150 on 4, because of how the rotation lines jobs up with CPUs.

**Simulator quirk:** with `-p -n 1` and a peek interval shorter than the run, `multi.py` crashes (`IndexError: Cannot choose from an empty sequence`). The single CPU tries to steal from "another" CPU, and there is none. Use `-P 0` with one CPU.

## Q9

> Finally, feel free to just generate random workloads and see if you can predict their performance on different numbers of processors, cache sizes, and scheduling options. If you do this, you'll soon be a multi-processor scheduling master, which is a pretty awesome thing to be. Good luck!

Random workloads with 4 jobs (`-s SEED -j 4`, run time and working set chosen at random; `-P 10` for per-CPU, `-P 0` with one CPU):

| seed (jobs run:wset) | CPUs | `-M 100` central | `-M 100` per-CPU | `-M 200` central | `-M 200` per-CPU |
|---|---|---|---|---|---|
| 1 (10:160, 70:50, 40:80, 60:150) | 1 | 180 | 180 | 160 | 160 |
| | 2 | 90 | 80 | 80 | 65 |
| | 4 | 60 | 60 | **45** | 55 |
| 2 (90:180, 0:10, 80:140, 60:60) | 1 | 231 | 231 | 217 | 217 |
| | 2 | 120 | 110 | 110 | **80** |
| | 4 | 90 | 90 | **50** | 70 |
| 3 (20:100, 30:120, 60:10, 0:160) | 1 | 101 | 101 | 76 | 76 |
| | 2 | 56 | 65 | 51 | 55 |
| | 4 | 45 | 40 | 40 | 40 |

**A worked prediction (seed 1, 4 CPUs, `-M 200`, central queue):** with 4 jobs on 4 CPUs, every job gets its own CPU, and all working sets fit in a 200-unit cache. Job 0 (10 units) finishes cold at tick 10. The 3 remaining jobs are then requeued and taken by CPUs 0, 1, 2, so each of them **moves** to a new, cold CPU once. They stay there afterwards, because the order of the queue no longer changes. So each of jobs 1–3 runs 20 cold ticks (10 on its first CPU, 10 on its second) before running at rate 2:

- job 2: 20 + 20/2 = 30
- job 3: 20 + 40/2 = 40
- job 1: 20 + 50/2 = **45**

The simulator agrees: **45**.

General lessons from these runs:

1. **A bigger cache only helps the jobs that fit.** Seed 1's job 0 (160) and seed 2's job 0 (180) never fit in a 100-unit cache. Worse, every time such a job finishes warming up, it flushes the whole cache.
2. **More CPUs help the most when every job can have its own CPU and cache** (the super-linear effect of Q7), e.g. seed 2: 217 → 50 with 4 CPUs and `-M 200`.
3. **Per-CPU queues help when jobs outnumber CPUs** (2 CPUs: 80 vs 90, 65 vs 80, 80 vs 110). They can hurt when CPUs are plentiful, because idle CPUs steal jobs that were already warm elsewhere (4 CPUs, `-M 200`: 55 vs 45, 70 vs 50).
4. **Simulator quirk:** random jobs can have a run time of 0 (seed 2 job 1, seed 3 job 3). Such a job still occupies a CPU for one tick, which is why seed 2 on one CPU with `-M 100` takes 231 ticks for 230 units of work.